

Projeto Prático: Desenvolvimento de um Sistema de Perguntas e Respostas Baseado em RAG

1. Objetivo Geral

O objetivo deste projeto é desenvolver um sistema completo de Perguntas e Respostas ou Chat Inteligente fundamentado na arquitetura Retrieval-Augmented Generation (RAG). O sistema deverá integrar a pipeline completa de RAG, o uso de APIs para acesso a modelos de linguagem de grande porte (LLMs), um banco vetorial para recuperação semântica, uma interface interativa desenvolvida em Streamlit ou Dash e a funcionalidade de upload e processamento de documentos.

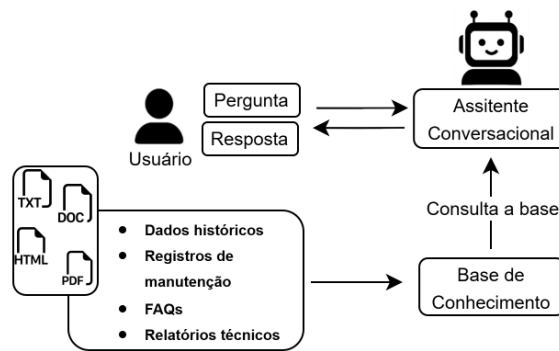


Figura 1: Visão do sistema

O sistema permitirá que um usuário envie um documento e realize perguntas sobre seu conteúdo, recebendo respostas fundamentadas nos trechos recuperados do próprio documento.

2. Descrição do Problema

Modelos de linguagem possuem amplo conhecimento pré-treinado, porém não têm acesso direto a documentos específicos do usuário. A arquitetura RAG resolve esse problema ao combinar recuperação de informações com geração de texto.

Neste projeto, os vocês deverão implementar um sistema capaz de receber um ou mais documentos enviados pelo usuário, processar e indexar esses documentos, recuperar trechos semanticamente relevantes para cada pergunta, gerar respostas utilizando um LLM acessado via API condicionado aos trechos recuperados e apresentar os resultados em uma interface interativa.

3. Arquitetura Conceitual do Sistema

A arquitetura do sistema deverá conter os seguintes componentes: ingestão de dados, segmentação do texto, geração de embeddings, armazenamento em banco vetorial, recuperação por similaridade, construção do prompt, geração de resposta via API de LLM e interface de usuário. A pipeline do sistema utilizando RAG é demonstrada na Figura 2.

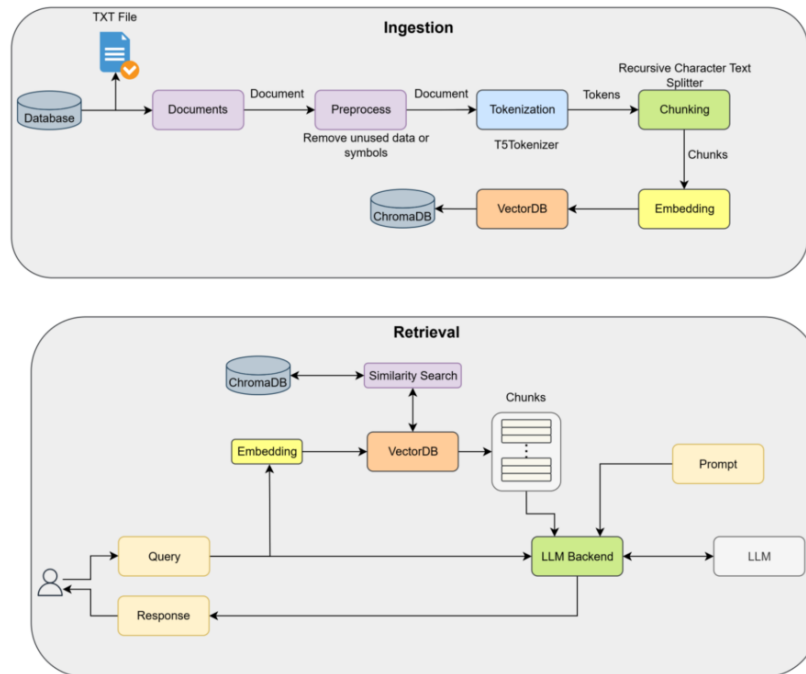


Figura 2: Pipeline do RAG

Cada um desses elementos deverá ser implementado de forma explícita, evidenciando a compreensão do fluxo completo da arquitetura RAG.

4. Etapas do Desenvolvimento

4.1 Ingestão de Dados

O sistema deve permitir o upload de documentos em um ou mais formatos como PDF e TXT. O texto deverá ser extraído e submetido a um pré-processamento básico, incluindo remoção de caracteres inválidos, normalização de espaços e organização do texto em formato contínuo. O resultado desta etapa deve ser um texto bruto pronto para segmentação.

4.2 Segmentação do Texto

Como modelos de embeddings e LLMs possuem limite de tokens, o texto deve ser dividido em segmentos menores. Definir uma estratégia de segmentação baseada em tamanho fixo ou em tamanho com sobreposição entre segmentos. A escolha dos parâmetros adotados deve ser tecnicamente justificada.

4.3 Geração de Embeddings

Cada segmento deverá ser convertido em vetor numérico utilizando uma API de embeddings. É obrigatório o uso explícito de uma API externa para geração de embeddings. Os vetores devem ser armazenados juntamente com seus respectivos segmentos.

4.4 Banco Vetorial

Os embeddings gerados deverão ser armazenados em um banco vetorial, como FAISS ou ChromaDB. O banco deve permitir busca por similaridade vetorial e suportar recuperação eficiente dos k segmentos mais relevantes.

4.5 Recuperação de Informação

Quando o usuário fizer uma pergunta, o sistema deverá gerar o embedding da pergunta, realizar busca no banco vetorial, recuperar os k segmentos mais similares e retornar os segmentos juntamente com seus respectivos scores de similaridade. O valor de k deve ser parametrizável.

4.6 Construção do Prompt

Os segmentos recuperados deverão ser incorporados em um prompt estruturado. O prompt deve conter instruções claras ao modelo, o contexto recuperado, a pergunta do usuário e uma orientação explícita para evitar respostas fora do contexto fornecido.

4.7 Geração de Resposta via API

A resposta deverá ser gerada utilizando um modelo de linguagem acessado por API. É obrigatório o uso explícito de API de LLM, com definição de parâmetros como temperatura e limite de tokens. O sistema deve tratar exceções e lidar com possíveis falhas de requisição.

5. Interface do Usuário

A aplicação deve ser desenvolvida utilizando Streamlit ou Dash. A interface deve conter área para upload de documento, botão para indexação, campo para envio de perguntas, exibição da resposta gerada, histórico de conversa e visualização opcional dos trechos recuperados.

O sistema pode (não é obrigatório) operar tanto em modo de perguntas isoladas quanto em modo de chat contextual com manutenção de histórico.

6. Parte Experimental

Entre os experimentos sugeridos estão a variação do tamanho dos segmentos, a variação do número de documentos recuperados, o uso ou não de sobreposição entre segmentos e a variação da temperatura do modelo. Isto é:

1. Variação do tamanho dos segmentos.
2. Variação do número de documentos recuperados (top-k).

3. Uso ou não de sobreposição entre segmentos.
4. Variação da temperatura do modelo.
5. Comparação entre diferentes modelos de LLM (se possível).

Cada experimento deve apresentar:

- Configuração adotada.
- Resultados observados.
- Análise crítica.

Cada experimento deve apresentar os achados em um compilado em forma de slides.

7. Links Úteis

- ABHYUDAY, Tejpal. *Retrieval-Augmented Generation (RAG): From Basics to Advanced*. Medium. Disponível em: <https://medium.com/@tejpal.abhyuday/retrieval-augmented-generation-rag-from-basics-to-advanced-a2b068fd576c>
- DRJULIJA. *What is Retrieval-Augmented Generation (RAG)?* Medium. Disponível em: <https://medium.com/@drjulija/what-is-retrieval-augmented-generation-rag-938e4f6e03d1>
- Data Science Collective. *From Data to Dialogue: Development of a RAG Chatbot for Fitness*. Medium. Disponível em: <https://medium.com/data-science-collective/from-data-to-dialogue-development-of-a-retrieval-augmented-generation-rag-chatbot-for-fitness-b9fbaf818ace>
- AIPlanet. *Evaluating Naive RAG and Advanced RAG Pipeline using LangChain v0.1.0 and RAGAS*. Medium. Disponível em: <https://medium.aiplanet.com/evaluating-naive-rag-and-advanced-rag-pipeline-using-langchain-v-0-1-0-and-ragas-17d24e74e5cf>
- Majumdar, Shoumik. *RAG Pipeline Implementation (RAG.py)*. GitHub. Disponível em: https://github.com/ShoumikMajumdar/RAG_pipeline/blob/main/RAG.py
- HKUDS. *RAG-Anything*. GitHub. Disponível em: <https://github.com/HKUDS/RAG-Anything>